Floorplanning for Embedded Multi-die Interconnect Bridge Packages

Chung-Chia Lee[1] and Yao-Wen Chang[1,2]
[1]Graduate Institute of Electronics Engineering, National Taiwan University, Taipei 106319, Taiwan
[2]Department of Electrical Engineering, National Taiwan University, Taipei 106319, Taiwan
cclee@eda.ee.ntu.edu.tw; ywchang@ntu.edu.tw

2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD) | 979-8-3503-2225-5/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICCAD57390.2023.10323609

***Abstract*—Modern heterogeneous integration requires dense IO interconnections among chips, such as CPU and memory, to facilitate bandwidth-aware packaging. The embedded multi-die interconnect bridge (EMIB) has attracted much attention recently by providing a high wiring density and low manufacturing cost. However, EMIB optimization must consider constrained wire orientations and crosstalk. This paper presents the first work on floorplanning for EMIB-based packaging. We first model the floorplanning problem for EMIB-based packaging. Based on a hybrid structure of transitive closure graphs and B*-trees, we present a novel simulated-annealing-based algorithm to efficiently generate the desired EMIB-aware floorplans. We employ maximum-spanning-tree-based partitioning and tree-based classification for already found partial topologies to search for desired solutions more efficiently. Experimental results show that our algorithm can significantly improve the area, total wirelength, and computation time compared with simulated annealing based on TCGs alone.**

I. Introduction

In modern heterogeneous integration, the power efficiency requirement for electrical transportation among chips continually increases the need for high memory bandwidth and dense IO interconnections [1]. Especially with the rapid growth of machine learning, high-performance computation of graphics processing units (GPUs) has become a primary way to reduce computational costs significantly. Therefore, the heterogeneous integration of GPUs and high-bandwidth memories (HBMs) requires much more technical support to achieve high-bandwidth transportation. The peak value of bandwidth density is the product of data lane density and data rate. For efficient computation on the processors requiring high-performance computing, high data rate and I/O interconnections are applied. Thus, bandwidth-aware packaging technologies, such as flip-chip ball grid arrays (FCBGAs) and silicon interposers, are also studied widely [2], [3].

The FCBGA is an elementary packaging technology used for high-density IO connections in the early years. However, with the rapid increase in the requirement for high bandwidth, technologies with only FCBGAs are insufficient to support high-density IO connections. Silicon interposers have become the leading bandwidth-aware packaging technologies for 2.5D and 3D IC fabrication [3], and there has been much research on silicon-interposer-based physical designs [4], [5], [6], [7]. However, silicon interposers have some drawbacks, such as their reticle size limitation and high manufacturing cost.

The embedded multi-die interconnect bridge (EMIB) emerges as a promising alternative to replace silicon interposers for high-bandwidth transportation. EMIBs create localized wire connections and use through-package vias to achieve much lower manufacturing costs than silicon interposers. However, EMIBs incur physical and crosstalk constraints, which can be seen as the combination of wire orientation and adjacency constraints. These constraints impose significant restrictions on the die positions, making the placement of EMIB-based multi-chip packaging (MCP) much more complex than silicon interposer-based ones.

A. Embedded Multi-die Interconnect Bridge

An EMIB is a bridge-based packaging that uses bridge-like structures to connect signal wires among dies, such as CoWoS-L [8] and FOCoS-B [9]. The EMIB uses multi-layer silicon bridges and embeds them within the top-two layers of an organic substrate [10], [11]. Mahajan and Sane [12] proposed the prototype of EMIB technologies, and Starkston *et al.* [13] integrated the silicon bridges into the organic substrate with low-density routing structure.

This work was partially supported by AnaGlobe, ASUS, Delta Electronics, Google, Maxeda Technology, TSMC, and NSTC of Taiwan under Grants NSTC 110-2221-E-002-177-MY3, NSTC 112-2218-E-002-033, and NSTC 112-2223-E-002-020.

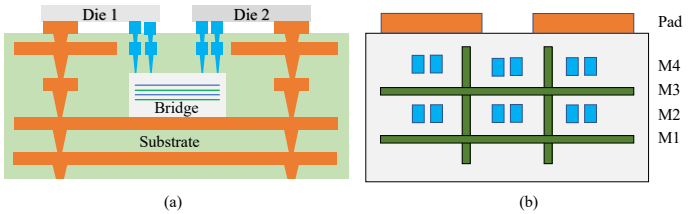


Figure 1. (a) The side view of a two-die package with a silicon bridge connection. Two dies (gray blocks) are placed under the organic substrate (green block), and die-to-bridge connections (blue links) and die-to-substrate connections (orange links) are used for package routing. (b) The cross-sectional view of a silicon bridge, where M2 and M4 are signal layers (blue blocks), and M1 and M3 are ground metals (green lines).

The multi-layer silicon bridge supports through-package vias with fine pitches and high wiring density, making high-density I/O links between two dies available to be routed locally. We can separate the package routing into the one in the silicon bridges and organic substrate, and the two types of routing do not interact. Figure 1(a) shows the side view of an EMIB-based package. We can route the connections with high wire density between two dies with through-package vias and silicon bridges. The vias between the dies and silicon bridges have finer pitches than those between the dies and the organic substrate. A silicon bridge has four metal layers and a pad layer labeled in Figure 1(b), where M1 and M3 are ground metals, and M2 and M4 are signal-routing layers. Every three signal wires are allocated a ground trace to reduce the impact of signal-to-signal crosstalk. Based on the EMIB attributes and the silicon bridge structure in [11], we consider one silicon bridge containing two signal layers with the same wire orientation. This configuration makes wire detours in a silicon bridge impossible, and the connections between two dies should be routed horizontally or vertically. Furthermore, with the narrow traces in the routing channels, there could be a surge in energy loss and crosstalk if a silicon bridge with a long reach is configured [14], [15]. Therefore, it is desirable to make two dies with bridge connections placed adjacently to reduce energy loss and crosstalk. With the capability of supporting high wiring density, it is appropriate to route dense IO connections in a silicon bridge. In contrast, other connections with lower wire density or die-to-PCB connections can be routed in the organic substrate.

EMIBs can outperform several features compared with silicon interposers [11]. Figure 2 shows the packaging technologies of EMIBs and silicon interposers. EMIBs insert a silicon bridge in the top-two layers of the organic substrate, while silicon interposers place dies under a piece of silicon multi-layer interposer. The assignment of the silicon bridges for EMIBs is based on the die positions and wire routes, and the bridge size is generally smaller than the die size. However, for silicon interposers, the area of the interposer is at least larger than the bounding area of the layout, making the layout restricted by the reticle size. Furthermore, silicon interposers use through-silicon vias (TSVs) to route signal wires between layers, which affects signal integrity (SI) and power integrity (PI) on wire connections. EMIBs use only small regions to place silicon bridges, and the organic substrate is manufactured by a polymeric material so that each IO connection can be routed without any interference between each other.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on August 20,2026 at 09:43:43 UTC from IEEE Xplore. Restrictions apply.

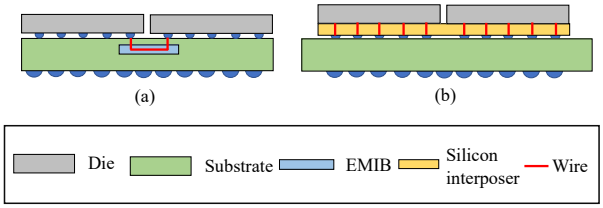


Figure 2. (a) EMIB and (b) silicon interposer packaging technologies.

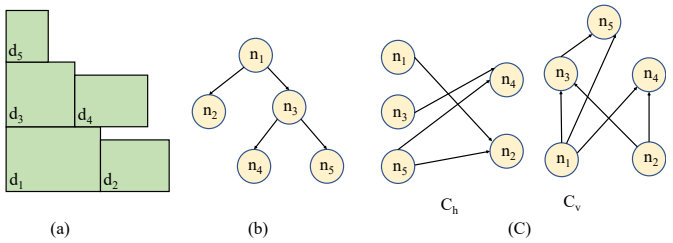


Figure 3. (a) A floorplan for five dies. (b) The corresponding B*-tree. (c) The corresponding TCG.

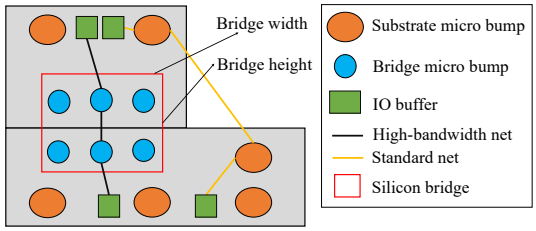


Figure 4. The wire connection model of standard nets and high-bandwidth nets.

B. Previous Works

Wire orientation and adjacency are the two main considerations for EMIB-aware floorplanning. The bus-driven floorplanning problem has similar wire orientation constraints. Bus assignment requires multiple dies to be aligned in a specific range and connect these dies with a multi-bend pin array. The feature of the EMIB wire orientation constraint is similar to the alignment feature of bus nets, which requires dies to be aligned either horizontally or vertically. Chen and Chang [16] inserted dummy nodes into a B*-tree, a topology representation for compacted floorplans with linear-time transformation [17], to legalize the layout accompanied by the inspection of illegal topologies. Ma and Yang [18] also inserted dummy nodes to legalize topologies but used the transitive closure graph (TCG), a topology representation for general floorplans [19], to represent the layout. As for the adjacency constraint, Wu *et al.* [20] proposed QB-trees to capture the adjacency of dies and hold the maximum separation constraint by look-ahead constraint checking. Tsao *et al.* [21] combined corner stitching, a data structure for accurate block neighboring information, and a B*-tree to handle various constraints.

C. Our Contributions

To our best knowledge, there is no published work for the floorplanning or placement of EMIB-based MCP. This paper formulates the floorplanning problem of EMIB-based MCP and minimizes the bounding area and wirelength with a specified aspect ratio constraint. We summarize our contributions as follows:

- We propose a hybrid structure, called *BT-trees*, with a B*-tree and a TCG to compact a partial layout and reduce computation time.
- We propose a maximum-spanning-tree-based partitioning scheme to find legal topologies with various structures efficiently.
- We propose tree-based classification for already found partial topologies to accelerate the search for desired solutions.
- We propose a novel simulated-annealing-based framework to efficiently explore feasible solutions.
- Experimental results show that our algorithm can significantly improve the area, total wirelength, and computation time compared with simulated annealing based on TCGs alone.

The remainder of this paper is organized as follows. Section II reviews B*-trees and TCGs and then formulates our problem. Section III presents our floorplanning flow and details each step. Section IV reports our experimental results. Finally, Section V concludes our work.

II. Preliminaries

In this section, we first review the B*-tree and TCG floorplan representations, present the wire connection model for the EMIB-based packaging, and formulate the EMIB-based floorplanning problem.

A. Review of the B*-tree

A B*-tree defines a floorplan with an ordered binary tree [17]. Let d_i be the i-th die, and x_i, y_i, and w_i be the x-coordinate, y-coordinate, and width of d_i, respectively. In a B*-tree, each x_i is determined directly by the tree structure. If a node n_i is the left child of n_j, then $x_i = x_j + w_j$. Otherwise, if n_i is the right child of n_j, $x_i = x_j$. On the other hand, each y_i is determined by the horizontal contour of the current floorplan. Each die is placed at the lowest possible location after its x-coordinate is decided. For example, the B*-tree in Figure 3(b) represents the floorplan in Figure 3(a). B*-tree node n_4 is the left child of n_3, so die d_4 is placed to the right boundary of d_3. Moreover, according to the horizontal contour given by n_1 and n_2, d_4 can be placed on top of n_1. A linear-time transformation from a tree to a layout can be achieved by using a doubly linked list to record the horizontal contour. Nevertheless, B*-trees have smaller solution space with incomplete adjacency information, which may complicate the legalization of physical constraints. As a result, work has been proposed to integrate B*-trees with other data structures to develop some hybrid data structures for various applications [20], [21].

B. Review of the Transitive Closure Graph

A transitive closure graph (TCG) [19] models a floorplan with a pair of constraint graphs (C_h and C_v) that represents the horizontally and vertically relative die positions. Figure 3(c) gives the TCG representation of the floorplan in Figure 3(a). The directed edges in C_h and C_v indicate the relative die positions. For example, there are edges $n_3 \rightarrow n_4$ and $n_5 \rightarrow n_4$ in C_h, so that the die d_4 should be right to d_5 and d_3. To avoid overlap among dies, every pair of dies must have exactly one edge in either C_h or C_v. Furthermore, if two edges $n_i \rightarrow n_j$ and $n_j \rightarrow n_k$ exist in one constraint graph, then $n_i \rightarrow n_k$ must also exist in the same constraint graph to hold transitive closure property.

C. Wire Connection Model for the EMIB-based Packaging

Let $D = \{d_1, d_2, ..., d_n\}$ be the set of n dies. There are two netlists, a standard netlist and a high-bandwidth netlist, which should be routed in the organic substrate and silicon bridge, respectively. We refer to the connection model of the nets mentioned in [22] and illustrate it in Figure 4. Every net in the standard and high-bandwidth netlist consists of two intra-die nets and one external net. An intra-die net connects an IO buffer and a micro bump in a die, and an external net connects two micro bumps on either substrate or a silicon bridge. The wire orientation of each net in the high-bandwidth netlist is confined to either vertical or horizontal direction because the silicon bridges should be placed vertically or horizontally, and wire detours are unavailable in them. For the width and height of each silicon bridge b_{ij} connecting die d_i and d_j, we let the height be constant value H_{bridge} and the width be W_{ij} to express the aspect of b_{ij}. If there are m high-bandwidth nets between d_i and d_j, and the maximum wiring density in the silicon bridges is c, then $W_{ij} = \frac{m}{c}$. Therefore, we can construct a bridge netlist $B = \{(b_{i,j}, W_{ij})\}$, which consists of bridge connections of d_i and d_j with the width value W_{ij} if there exists a high-bandwidth net connecting d_i and d_j. We will use the bridge netlist to hold the EMIB constraints.

D. Problem Formulation

With the die set, standard netlist, high-bandwidth netlist, and corresponding bridge netlist, the objective of the EMIB-aware floorplanning

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on August 20,2026 at 09:43:43 UTC from IEEE Xplore. Restrictions apply.

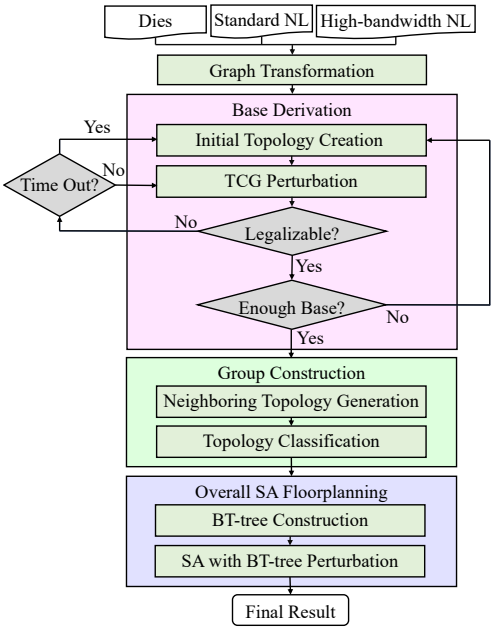


Figure 5. Our EMIB-aware floorplanning algorithm flow. (NL: Netlist)

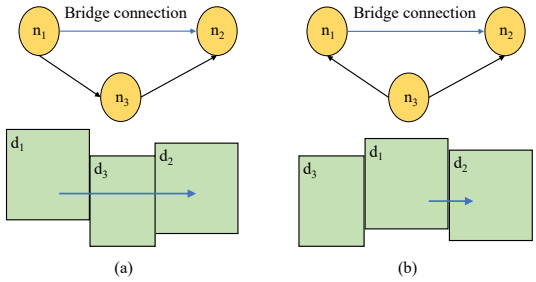


Figure 6. (a) Violation and (b) legality of the EMIB adjacency constraint.

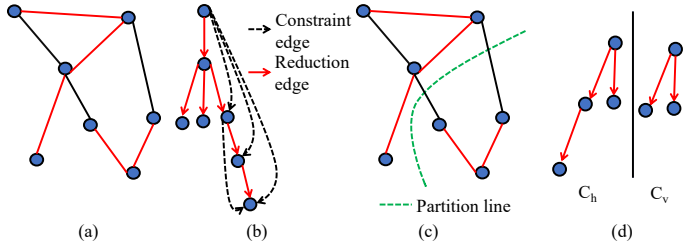


Figure 7. (a) A maximum spanning tree in an ECG. (b) A topology based on a maximum spanning tree. (c) Partitioning an ECG. (d) A topology created from maximum-spanning-tree-based partitioning.

problem is to optimize the floorplan with bridge assignments, which minimizes the bounding area and wirelength with a specified aspect ratio constraint. Furthermore, any pair of two dies d_i and d_j with a bridge connection should be placed adjacently and has sufficient overlap range W_{ij} to place a silicon bridge.

III. Proposed Algorithms

In this section, we first give an overview of the whole algorithm flow. Figure 5 shows our algorithm flow, which consists of the following four stages: (1) *Graph Transformation*, (2) *Base Derivation*, (3) *Group Construction*, and (4) *Overall SA Floorplanning*. At first, in Graph Transformation, we transform the die set and bridge netlist into some connected graphs and perform Base Derivation and Group Construction for each graph separately. In Base Derivation, we identify a few legal topologies for each connected graph using simulated annealing (SA) with TCGs. In Group Construction, we search for neighboring legal topologies of the identified topologies and classify them based on the similarity degree of their structures. Finally, in Overall SA Floorplanning, we merge the found topologies into a BT-tree, a B*-tree and TCG hybrid structure, to generate overall legal topologies and then implement SA to find the optimized floorplan.

A. Graph Transformation

At first, we transform the die set D and bridge netlist B into some connected graphs, which must have no bridge connection between each other. We call these EMIB-connected graphs ECGs and let each be ECG_i. When searching for the topology with the lowest cost, it is necessary to legalize every found topology to hold the EMIB constraints. Legalizing each ECG separately rather than an overall topology can reduce redundant time traversing the overall topology if we want to find legal but not optimized topologies. Therefore, we handle each ECG separately in Base Derivation and Group Construction to find the partial legal topologies of each ECG. Finally, we merge the results from the two stages to generate overall legal topologies in Overall SA Floorplanning.

B. Base Derivation

This stage aims to find some topologies and choose them as bases of the current ECG. The bases of an ECG are the legal topologies with distinct TCG structures. We want to find other legal topologies based on them by TCG perturbations in Neighboring Topology Generation, which is detailed in III-C1. We can enormously decrease the probability of producing identical topologies due to the significant structural difference among the initial topologies. We use TCGs as the topology representation because TCGs have a large solution space and complete geometrical information useful in legalizing the EMIB constraints.

1) Initial Topology Creation: In TCGs, an edge $n_i \rightarrow n_j$ is said to be a reduction edge if there is no other node n_k, such that two edges $n_i \rightarrow n_k$ and $n_k \rightarrow n_j$ exist. Otherwise, it is a closure edge. Based on the geometrical relation of closure and reduction edges illustrated in Figure 6, if there is a closure edge between two dies, it is impossible to place two dies adjacently because there must be a die between them. As a result, if there is a bridge connection between d_i and d_j, there must be a reduction edge connecting n_i and n_j in either C_h or C_v. We propose a maximum-spanning-tree-based method to construct various TCG topologies with sufficient reduction edges. The process is illustrated in Figures 7(a) and (b):

- Step 1: Find a maximum spanning tree (MST) in an ECG by performing Prim's algorithm [23].
- Step 2: Randomly choose a node to be the root of the MST, and let the edges in the MST be directed from the parent nodes to their children.
- Step 3: Add constraint edges to hold the transitive property of TCGs. For every n_i and n_j, if n_i is an ancestor of n_j, we add constraint edge $n_i \rightarrow n_j$ into the topology. If we choose the current topology to be in C_h (C_v), the remaining edges we have not added should be added in C_v (C_h).

This MST-based method ensures that at least $D_i - 1$ edges in the MST are reduction edges, where D_i is the number of dies in ECG_i. Furthermore, as in Figures 7(c) and (d), we can randomly partition ECG into two sub-graphs, place one sub-graph in C_h and the other in C_v, and perform the MST-based method on these two sub-graphs separately to create another initial topology. With the decision of partition lines, roots, and the direction of the remaining edges, we can generate various topologies with sufficient reduction edges. We will utilize this variability to find the bases of ECGs extensively. We give the time complexity of the MST-based partitioning and the corresponding proof as follows:

Theorem 1: The time complexity of the maximum-spanning-tree-based partitioning is $O(D^2)$, where D is the number of the dies.

Proof 1: At first, the partitioning can be performed in $O(D)$ time by randomly determining every node belongings of the two sides. After partitioning, Prim's algorithm takes $O(D^2)$ time to derive two MSTs. Since there must be exactly one edge between two dies in TCGs, the TCG construction from the two MSTs is in $O(D^2)$ time. Prim's algorithm and TCG construction take $O(D^2)$ time and dominate the overall time complexity.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on August 20,2026 at 09:43:43 UTC from IEEE Xplore. Restrictions apply.

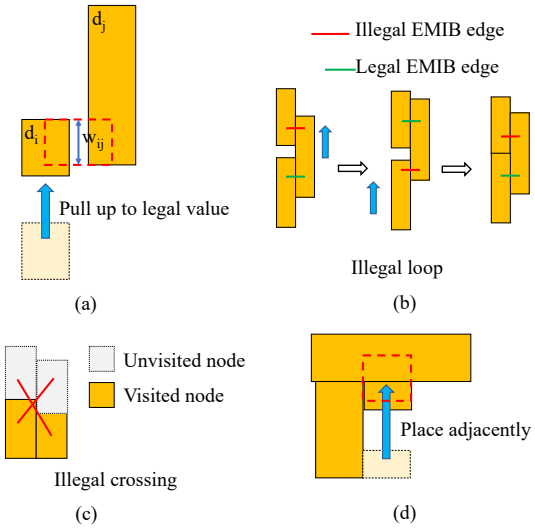


Figure 8. (a) EMIB edge legalization. (b) An illegal loop. (c) An illegal crossing. (d) Adjacency adjustment.

2) EMIB Legalization: This step is to adjust node values in TCGs such that for every b_{ij} in B, the overlap range of d_i and d_j should be larger than W_{ij}. We can conduct EMIB Legalization in C_h and C_v separately because node values of C_h (C_v) only affect vertical (horizontal) bridge connections. Initially, suppose that there is a reduction edge $n_i \rightarrow n_j$ in C_h (C_v) and a bridge connection between d_i and d_j. In that case, we add an undirected EMIB edge E_{ij} between n_i and n_j in C_v (C_h) to represent a bridge connection. In Figure 8(a), the legalization of an EMIB edge is to pull up the bottommost node to a strictly legal value. When visiting a node n_i with an EMIB edge E_{ij}, we skip this node and traverse other nodes until we visit n_j such that the values of n_i and n_j are determined, and thus we can legalize E_{ij}. When legalizing one EMIB edge, the process of the legalization may cause other legalized EMIB edges to become illegal again. We should legalize these edges until no new illegal edge appears. Two conditions make the topology infeasible to be legalized. The first condition is that there is no sufficient overlap range for connecting two or more dies, making the legalization infeasible to alter as in Figure 8(b). The other condition is that twisting EMIB edges occur, and legalizing them is impossible, as in Figure 8(c). After the traversal of TCGs, we adjust node values to place two dies with a bridge connection adjacently as in Figure 8(d). If the adjustment causes some EMIB edges to become illegal, we will discard the current topology and continue to find other legal topologies.

3) Bridge Overlap Adjustment: In a compacted floorplan, bridge overlaps may occur at the corner of the dies because every silicon bridge has an overlapped region on the connected dies. Figure 9(a) shows a floorplan with two bridges overlapping at the left-bottom corner of die D_3. To avoid bridge overlaps, we check the related positions of its connected dies for each die with multiple bridge connections and introduce overlap extensions on the top and bottom connected dies before EMIB Legalization. For example, in Figure 9(b), we check the corresponding TCG and find that D_2 and D_4 are at the left and right sides of D_3, respectively. We introduce a left and right overlap extension to shrink the legal region of D_1 by temporarily increasing the width of D_1 and the required overlap between D_1 and D_3 by the left overlap extension plus the right one. After deriving the temporary width and required overlap, we perform EMIB Legalization to produce the TCG without bridge overlaps.

4) Base Derivation Process: The kernel process of Base Derivation is shown in Figure 10(a). We conduct the MST-based partitioning to create a topology T_i with sufficient reduction edges and check whether this topology is legal. If it is legal, we let it be a base σ_i and put it into the base set. After deriving a base, we conduct the MST-based partitioning again to find other bases until a sufficient number of bases are obtained. Otherwise, we perform cycling of TCG perturbations and EMIB Legalization on this topology to find a legal topology.

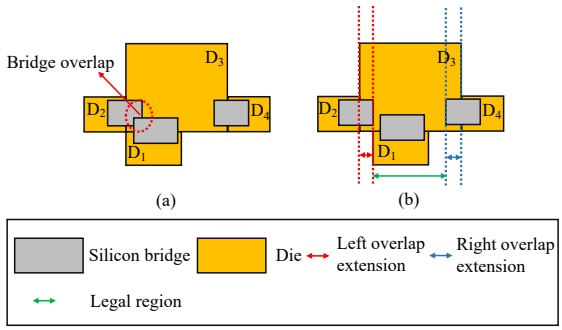


Figure 9. (a) The floorplan with a bridge overlap. (b) The adjusted floorplan derived from overlap extension.

5) Legality-optimized SA: The cycling of TCG perturbations and EMIB Legalization is formulated as an SA-based optimization. We define a cost C_{legal} to represent the degree of legality:

$$C_{legal} = \alpha_c P_c(T_i) + \beta_l P_l(T_i), \tag{1}$$

where $P_c(T_i)$ is the proportion of closure edges and EMIB edges in T_i, and $P_l(T_i)$ is the proportion of illegal EMIB edges and EMIB edges in T_i. We approximate the number of illegal EMIB edges to be the total number of EMIB edges minus the current legal EMIB edges. We can regard $P_l(T_i)$ as the lower bound for the degree of legality. α_c and β_l are the weights of C_{legal}. With the cost setting, the SA will guide topologies to approach a legal TCG structure. It also converges rapidly due to the suitable initial topologies from Initial Topology Creation.

However, there may be an initial topology derived from the MST-based partitioning with the TCG structure far from legal topologies, which makes SA consume much time in searching for legal topologies based on it. To avoid this situation, we let $P_c(T_i)$ be a baseline for every topology T_i. If $P_c(T_i) \geq l_c$, where l_c is a constant value and $0.5 < l_c < 1$, we alter the current legality-optimized SA and perform MST-based partitioning to explore other legal topologies based on a new initial topology.

C. Group Construction

We conduct Neighboring Topology Generation to find neighboring legal topologies δ_j^i from σ_i and construct a tree-based structure to express the similarity degree between each already found δ_j^i.

1) Neighboring Topology Generation: To find legal topologies, we implement Legality-optimized SA on each σ_i. The process of Neighboring Topology Generation is shown in Figure 10(b). For each σ_i, there is a base group set G_i. We implement legality-optimized SA to find a legal topology δ_j^i and insert it into G_i. We continue this process until the SA converges. We define a cost $C_{neighbor}$ to express the preference of the desired topologies:

$$C_{neighbor} = \alpha_x P_x(\sigma_i, T_j) + \beta_y P_y(\sigma_i, T_j) + \gamma_s P_s(\sigma_i, T_j), \tag{2}$$

where $P_x(\sigma_i, T_j)$ and $P_y(\sigma_i, T_j)$ is the width and height ratio of σ_i and T_j, respectively. α_x, β_y, and γ_s are the weights of $C_{neighbor}$. We choose the width and height as the cost parameters because we can compress the height of overall topologies with a horizontal contour but cannot compress the width, which will be mentioned in Section III-D. We set width and height as the cost parameters to discard the topologies with large widths and let α_x be larger than β_y. $P_s(\sigma_i, T_j)$ is the similarity force defined as follows:

$$P_s(\sigma_i, T_j) = \frac{|SD(\sigma_i, T_j) - S_e|}{1 - S_e}, \tag{3}$$

where $SD(\sigma_i, T_j)$ is the similarity degree of topology σ_i and T_j, and S_e is the expected similarity degree of every base. We approximate the similarity degree of two topologies in $O(D)$ time, where D is the die size, based on each node's top, right closest nodes and its orientation without checking every constraint edge. Figure 11(a) illustrates the interactions between bases and similarity force. The similarity force guides σ_i to be transformed to a topology with an S_e similarity degree compared with σ_i. The absolute value of the difference of the similarity

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on August 20,2026 at 09:43:43 UTC from IEEE Xplore. Restrictions apply.

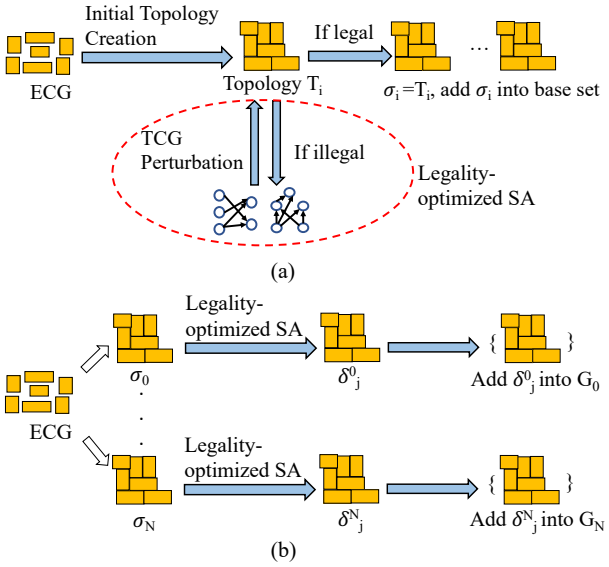


Figure 10. The process of (a) Base Derivation and (b) Group Construction.

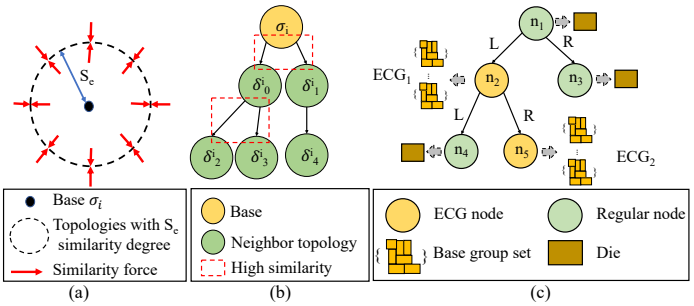


Figure 11. (a) A base and the corresponding similarity force. (b) A similarity tree with $N = 2$. (c) An example of BT-tree configuration.

degree in Equation (3) avoids searching for the topologies far from σ_i. If we find a legal topology T_j during legality-optimized SA of σ_i, we let δ_j^i be T_j and insert it into G_i. Because we record all found legal topologies during legality-optimized SA, similarity force can help us to obtain the neighboring legal topologies of every base. Furthermore, bases have distinct structures between each other, so we have less probability of recording identical topologies between two base group sets.

2) Topology Classification: We construct a similarity tree to express the related similarity degrees of the topologies in each base group set. For each base group set G_i, we choose σ_i as the root of the similarity tree and let σ_i be the first node for later operation. We describe the construction of the similarity tree in the following steps:

- Step 1: Compute the similarity degrees between the current node and all the topologies we have not connected to the tree.
- Step 2: Choose the topologies with the top N largest similarity degrees to be the children of the current node, where N is a positive integer based on the number of the topologies.
- Step 3: Choose the new current node in the similarity tree in breath-first order. Repeat Steps 1 and 2 until every topology is connected to the tree.

For example, Figure 11(b) is a similarity tree with $N = 2$. δ_0^i and δ_1^i have the top two largest similarity degrees of σ_i, so we let δ_0^i and δ_1^i be the children of σ_i. Among the remaining topologies, δ_2^i and δ_3^i have the top two largest similarity degrees of δ_0^i, so we let δ_2^i and δ_3^i be the children of δ_0^i. δ_4^i is the only one left, so we directly let it be the child of δ_1^i. With this construction, we can order the topologies with similar structures closely. This feature can help us change one topology to one with a slightly different structure, which we set as a movement of perturbations in Overall SA Floorplanning.

We give the time complexity of the tree-based classification and the corresponding proof as follows:

Theorem 2: The time complexity of the tree-based classification is $O(DM \lg M)$, where D is the die size, and M is the number of the topologies.

Proof 2: As mentioned in Section III-C1, we can compute the similarity degree of two topologies in $O(D)$ time. To connect the nodes to the root of the similarity trees, searching the top N similar topologies of the root takes $O(DM \lg N)$ time by maintaining the N-sized sort list for the M topologies. After finishing the node connection of the root, the number of the remaining topologies becomes $M - N$, so the time complexity of the next i-th node connection is $O(D(M - iN) \lg N)$. We have the overall time complexity as follows:

$$O(D(\sum_{i=0}^{\left\lceil \frac{M}{N} \right\rceil} C_i M - \sum_{i=1}^{\left\lceil \frac{M}{N} \right\rceil} iC_i N) \lg N), \tag{4}$$

where C_i is the coefficient of the time complexity of the i-th node connection. We let $N = aM$, where a is a constant value and $0 < a < 1$. Thus, the overall time complexity becomes

$$O(DM(C_0 + C_i \sum_{i=1}^{\left\lceil \frac{1}{a} \right\rceil} (1 - ia))(\lg a + \lg M)). \tag{5}$$

Since $C_0 + C_i \sum_{i=1}^{\left\lceil \frac{1}{a} \right\rceil} (1 - ia)$ is constant and $\lg M$ dominates $\lg a$, the overall time complexity becomes $O(DM \lg M)$.

D. Overall SA Floorplanning

We utilize the legal topologies from the previous two stages to construct and optimize the overall floorplan. We present a new hybrid structure of a B*-tree and TCGs (BT-tree, for short) to pack ECGs together and transform them into an overall EMIB-aware floorplan.

1) BT-trees: In a BT-tree, we use a B*-tree as the tree structure and TCGs as the topologies in the nodes. There are two node types, ECG nodes and regular nodes. An ECG node contains the base group sets of an ECG, and a regular node represents a die as the nodes in B*-trees. The function of the edges in BT-trees is the same as in B*-trees. Figure 11(c) shows an example of BT-trees, where n_1 represents a die without any bridge connection, and n_2 represents the dies in ECG_1 and contains the base group sets of ECG_1. We choose a topology in the sets as the initial topology of n_2 and change the topology by the perturbations of BT-trees, which will be explained in Section III-D3. BT-trees combine the features of B*-trees and TCGs. We can fast generate the overall floorplans with B*-trees and legalize the EMIB constraints with TCGs.

2) Packing Scheme: Figure 12 shows the packing scheme of ECG nodes consisting of the following four steps:

- Step 1: Determine the x-coordinate of the compacted position in the same way as Section II-A. The x-coordinate of the dies in the topology has been determined during EMIB Legalization conducted in Section III-B or Section III-C1.
- Step 2: Find the range segments that intersect the x range of the dies. To express the height difference below the ECG node, we transform each range segment into a dummy node and then connect the dummy node to the source node n_s of C_v and the nodes in C_v whose corresponding dies intersect the range segment. For example, the x interval of d_8 intersects the range of the dummy node S_2, so we add two constraint edges $n_s \rightarrow n_{s_2}$ and $n_{s_2 \rightarrow n_8}$ in C_v, where n_s is the source node of C_v, and n_{s_2} is the dummy node of contour segment S_2.
- Step 3: Perform EMIB Legalization on C_v and compute the y-coordinate of each die.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on August 20,2026 at 09:43:43 UTC from IEEE Xplore. Restrictions apply.

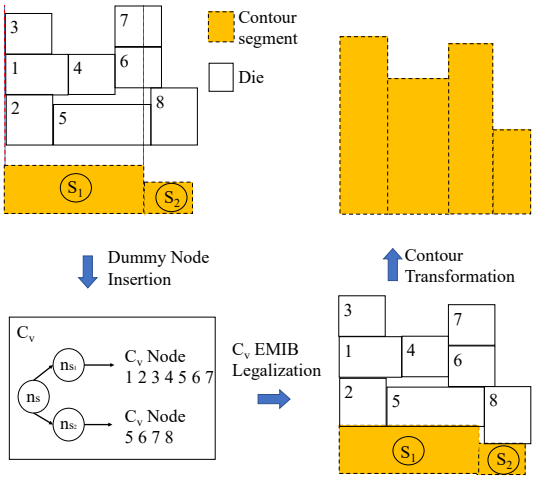


Figure 12. The BT-tree packing.

- Step 4: Update the horizontal contour and continue to pack the other nodes in the BT-tree.

The following theorem ensures the validity of our packing scheme:

Theorem 3: The BT-tree packing scheme always produces a floorplan without violating any EMIB constraint.

Proof 3: When placing the topology of an ECG node under the horizontal contour, we perform EMIB Legalization to compact the topology in the vertical direction. The EMIB Legalization will not alter the topologies of the ECG nodes that have been compacted on the floorplan since ECGs are independent. If there is no valid compacted floorplan, the legalization process will terminate at a legal solution where the topology is directly placed on the topmost position of the range segments in the horizontal contour by moving the compacted dies upwards. Therefore, each EMIB-aware topology can be packed together without violating the EMIB constraints.

3) Perturbation: We apply the following five perturbation operations on our BT-trees:

- Swap: Swap two nodes.
- Delete and Insert: Delete one node and insert it into the bottom of another node.
- Rotate: Rotate the current node 90 degrees if the node is a regular node; otherwise, rotate 180 degrees.
- Slight Transform: Change the topology of the current ECG node into another topology in the same base group set.
- Large Transform: Change the topology of the current ECG node into another topology in a different base group set.

Swap, Delete and Insert, and Rotate are the basic operations of B*-trees, except that we rotate an ECG node 180 degrees because there may be a topology in some base group set identical to the topology after rotating 90 degrees. Rotating 180 degrees can change the current topology from a left-compacted topology to a right-compacted topology. Slight Transform and Large Transform are topology-aware movements. For Slight Transform, we utilize the similarity tree explained in Section III-C2 to find a neighboring legal topology with a similar structure.

For example, in Figure 13(a), we start at the current topology δ_1^i, go top-down or bottom-up K times, and choose the topology we finally reach as the next topology, where K is a random positive integer in the range [1, 3]. We let $K = 2$, and the topologies in the region bounded by the red dashed line are the topology candidates which may be selected as the next topology. The time complexity of Slight Transform is $O(1)$ since we traverse the similarity trees in at most K steps, and K is a constant integer. For Large Transform, we change from the current base group set to another base group set and randomly choose a topology in it as the next topology for a large transformation of the topology structure. In Figure 13(b), we start at δ_1^i and choose the similarity tree with base σ_j as the tree with the next topology. Finally, we choose δ_5^j as the next topology. The time complexity of Large Transform is also $O(1)$ since we randomly choose a base in the

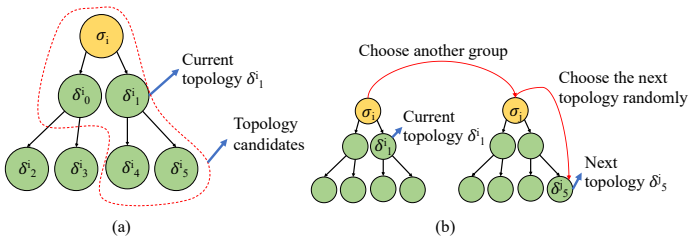


Figure 13. Two topology-aware movements starting from δ_1^i. (a) Slight Transform with $K = 2$. (b) Large Transform to δ_5^j.

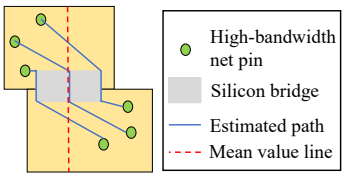


Figure 14. An example of EMIB assignment with a vertical wire orientation.

base set and transform the current topology to a random topology in the corresponding similarity tree of the selected base.

Theorem 4: The time complexity of Slight Transform and Large Transform is $O(1)$.

4) Cost Function: The cost function of Overall SA Floorplanning is defined as follows:

$$C_{overall} = \alpha A + \beta WL + \gamma(R - R^*)^2, \tag{6}$$

where A, WL are normalized bounding area and half-parameter wirelength, respectively, and $(R - R^*)^2$ is the outline cost function used for the fixed-outline constraint proposed in [16].

5) Silicon Bridge Assignment: We determine the position of a silicon bridge by the mean value of the x-coordinate or y-coordinate of high-bandwidth nets according to the wire orientations. In Figure 14, we derive the mean value line of the pins of the high-bandwidth nets and place a silicon bridge in the position symmetric to the mean value line. If the silicon bridge is out of the range of the overlap region, then we place it at the boundary of the overlapping region closest to the mean value line. We approximate the connection path of each high-bandwidth net by a three-segment net consisting of two pin-to-bridge nets and one inter-bridge net to compute the wirelength cost efficiently.

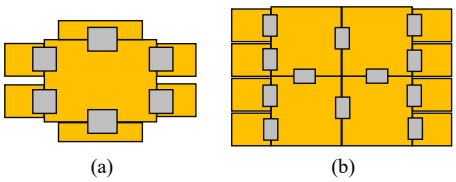


Figure 15. (a) Basic Stratix10 layout. (b) Basic Co-EMIB layout. The yellow and gray blocks are dies and silicon bridges, respectively.

Table I
ATTRIBUTES OF THE TESTCASES. THE NUMBER OF DIES (# DIE), TOTAL NUMBER OF BRIDGE CONNECTIONS (# BRIDGE), THE MAXIMUM NUMBER OF BRIDGE CONNECTIONS AMONG ALL ECGS (MAX # BRIDGE), THE TOTAL NUMBER OF ECGS (# ECG), AND NUMBER OF CONSTRUCTED BT-TREE NODES (# BT NODE) ARE LISTED IN THE TABLE.

Testcase	# Die	# Bridge	Max # Bridge	# ECG	# BT Node
Stratix10+4b	10	5	5	1	5
Stratix10+2E	6	7	7	1	1
CO0B	12	8	3	4	4
CO1B	12	9	5	3	3
CO2B	12	10	8	2	2
CO3B	12	11	11	1	1
CO4B	12	12	12	1	1

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on August 20,2026 at 09:43:43 UTC from IEEE Xplore. Restrictions apply.

Table II
THE COMPARISON OF THE BOUNDARY AREA (BA), HALF-PARAMETER WIRELENGTH (WL), ASPECT RATIO (AR), CPU TIMES (SEC) (TIME(S)) AND THE FLOORPLAN COST (COST) FOR SA-TCG AND OUR WORK.

Testcase	SA-TCG					Ours				
	BA	WL	AR	Time(s)	Cost	BA	WL	AR	Time(s)	Cost
Stratix10+4b	422500	67838	1.00	8.6	0.79	390000	61470	0.92	3.6	0.72
Stratix10+2E	385000	76452	1.27	22.1	0.75	357500	77359	0.84	6.5	0.71
CO0B	936000	113986	0.68	671.4	0.65	810000	96967	1.00	8.8	0.58
CO1B	942500	130501	0.48	3337.3	0.66	800000	111840	0.80	6.9	0.56
CO2B	990000	134732	0.82	3600.0	0.47	800000	129125	1.25	11.8	0.40
CO3B	990000	150203	1.22	3600.0	0.54	945000	149584	1.17	14.7	0.51
CO4B	-	-	-	3600.0	-	900000	163569	1.11	682.8	0.88
Comp.	-	-	-	156.98	1.11	-	-	-	1.00	1.00

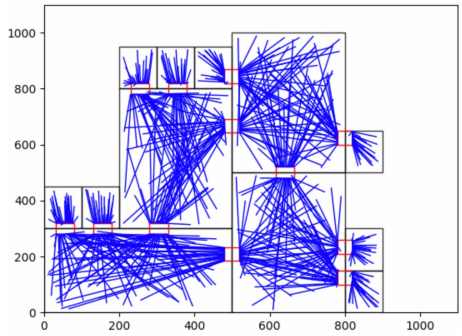


Figure 16. Our floorplan result of CO4B. Silicon bridges (red blocks) are placed between the overlapping region of two dies (black blocks), and high-bandwidth nets (blue lines) are connected to their corresponding silicon bridges. Standard nets are not shown for clear presentation.

IV. EXPERIMENTAL RESULTS

The proposed algorithms were implemented in C++ language on a 2.9GHz AMD Ryzen Threadripper 3990X Processor with 126GB memory. Because there is no previous work on the physical design of EMIB-based packaging, we referred to the package layouts presented in Intel's publication and designed seven testcases. The attributes of the testcases are shown in Table I. Figure 15 shows two basic layouts of our benchmarks. The first and second testcases are the revisions of Intel's Stratix 10 [24], and the third to seventh testcases are the revisions of Intel's published packages using Co-EMIB technologies [25]. We set the cost function parameters in Overall SA Floorplanning with $\alpha = 0.8$, $\beta = 0.1$, $\gamma = 0.1$, and $R* = 1$.

We performed four experiments: We first compared our work with SA based on TCGs alone (SA-TCG, for short). Next, we tested the effectiveness of MST-based partitioning and tree-based classification. Finally, we performed linear regression to derive the empirical time complexity of our algorithm and test the scalability.

A. Comparison with TCG-based SA Floorplanning

In the first experiment, we implemented SA-TCG by directly solving the overall floorplanning without clustering, so we need to perform EMIB Legalization for every generated topology. To accelerate the exploration of legal topologies, we applied the legality cost function mentioned in Section III-B4 to prompt the current topology to approach a legal topology. Table II shows the results. We labeled the bounding area, wirelength, and aspect ratio with BA, WL, and AR, respectively. We let SA-TCG and our work take the first legal topology from our work as the normalization base such that the cost comparison would be fair. We ran SA-TCG and our work for one hour and recorded the final cost. For the testcase CO2B and CO3B, SA-TCG could not converge in one hour, and SA-TCG could not find any legal topology for CO4B. Although the attributes of CO4B are similar to those of CO3B, CO4B has one additional bridge connection between center dies in Figure 15(b), making the solution space of CO4B much smaller than that of CO3B. CO4B has only one ECG, meaning that the exploration time of the legal topologies and the quality of the already found topologies dominate the final results. Our SA-based topology-searching framework could find legal topologies and optimize the cost effectively. Figure 16 gives our floorplan of CO4B. For the other testcases, our work achieved roughly an 11% improvement in the cost and converged in 12 minutes. Furthermore, we compared the convergence speed on the first to fourth testcases because SA-TCG can converge in one hour on these testcases. Our convergence speed is 140 times faster than SA-TCG on average.

Table III
THE COMPARISON OF CPU TIMES (SEC) (TIME(S)) OF BASE DERIVATION FOR OUR WORK WITHOUT MST-BASED PARTITIONING (NO MST-PARTITIONING) AND WITH MST-BASED PARTITIONING (MST-PARTITIONING). BASE DERIVATION ONLY DERIVED ONE BASE ON CO3B* AND CO4B*.

Testcase	No MST-Partitioning	MST-Partitioning
	Time(s)	Time(s)
CO0B	0.0023	0.0018
CO1B	0.0146	0.0060
CO2B	2.2712	0.2183
CO3B*	9.2560	0.3024
CO4B*	3063.8700	88.6229
Comp.	34.4965	1.0000

B. Effectiveness of MST-based Partitioning

In the second experiment, we tested the effectiveness of MST-based partitioning. We only performed Base Derivation to compare the speed of deriving the legal topologies. We compared the difference between Base Derivation without and with MST-based partitioning and experimented on the Co-EMIB testcases. We collected ten bases for CO0B, CO1B, and CO2B, and one for CO3B and CO4B since the work without MST-based partitioning could not find ten distinct bases in one hour for CO3B and CO4B. Table III shows the CPU time of Base Derivation of each testcase. For each testcase, we could finish Base Derivation more quickly. For CO3B, the work without the MST-based partitioning could find a legal topology in ten seconds. However, it could not find the other nine distinct topologies in one hour, meaning that the fixed initial topologies are difficult to be transformed into other legal topologies. For CO4B, finding the first legal topology took roughly 51 minutes, and we could not find the other legal topologies in one hour. Our MST-based partitioning speeds up the process of finding the legal topologies and effectively guides SA to find distinct ones.

C. Effectiveness of Tree-based Classification

In the third experiment, we tested the functionality of tree-based classification on CO0B, CO1B, and CO2B. Based on the cost variation in Figure 17, tree-based classification can make Overall SA Floorplanning converge more quickly and find the desired floorplan with a smaller cost. The experimental results show the effectiveness and efficiency of our algorithm.

D. Empirical Time Complexity and Scalability

In the fourth experiment, we performed linear regression to derive the empirical time complexity of our algorithm and examine the scalability. Legal topology exploration and the convergence speed of searching for the optimized solution are the two bottlenecks of the EMIB-aware floorplanning problem. We cannot directly use the testcases in the first experiment since the layout structures of the testcases are distinct, making the comparison of the die number and runtime unreasonable. To create new testcases and consider these two factors simultaneously, we let the first testcase consist of two ECGs, and each ECG is the basic structure of Stratix 10. Then, we created the $(i+1)$-th testcase by incrementally increasing the ECG structure complexity and the number of BT-nodes, as shown in Figure 18. D_i, N_i, and E_i denote the die number, the node number in each ECG, and the number of ECGs in the i-th testcase, respectively. The yellow-colored nodes in the ECGs of the $(i+1)$-th testcase indicate the newly added dies. We duplicated one ECG from the i-th testcase and added a new die in each ECG. We ran our algorithm on these testcases and recorded their runtime. The runtime is plotted as a function of the die number in Figure 19 (in log-log scale). According to the derived regression line labeled in Figure 19, the empirical time complexity of our algorithm is $O(D^{1.75})$, indicating the effectiveness in empirical

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on August 20,2026 at 09:43:43 UTC from IEEE Xplore. Restrictions apply.

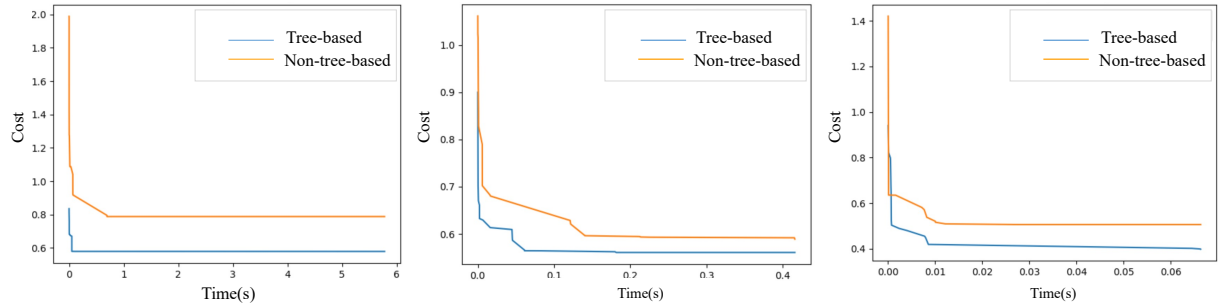


Figure 17. The convergence graphs of (a) CO0B, (b) CO1B, and (c) CO2B

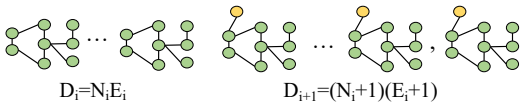


Figure 18. (a) The i-th testcase. (b) The $(i+1)$-th testcase.

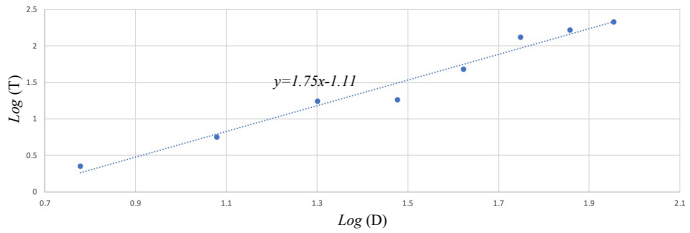


Figure 19. The log-log plot of the CPU time in seconds (T) and die number (D).

evaluation. Although the eighth testcase contains roughly 80 dies and 9 ECGs, our work could still optimize the floorplan cost in 3 minutes, revealing the super scalability of our work.

V. Conclusions

To our best knowledge, our algorithm is the first work for the EMIB-aware Floorplanning problem and also the first work for the physical design of EMIB-based packaging. We have proposed a novel SA-based framework to search for legal topologies in limited solution space efficiently. Based on the found topologies, our algorithm can generate EMIB-aware floorplans with BT-trees efficiently. Furthermore, we have implemented maximum-spanning-tree-based partitioning, tree-based classification, and topology-aware movement to improve the optimization quality and convergence speed. Experimental results have shown that our algorithm can significantly improve floorplan quality and CPU time compared with SA based on TCGs alone.

References

[1] D. C. Daly, L. C. Fujino, and K. C. Smith, "Through the looking glass—the 2018 edition: trends in solid-state circuits from the 65th ISSCC," *IEEE Solid-State Circuits Magazine*, vol. 10, no. 1, pp. 30–46, 2018.

[2] D. Y. Chong, B. Lim, K. Rebibis, S. Pan, S. Krishnamoorthi, R. Kapoor, A. Sun, and H. Tan, "Development of a new improved high performance flip chip BGA package," in *Proc. of ECTC*, Las Vegas, NV, USA, August 2004, pp. 1174–1180.

[3] M. Sunohara, T. Tokunaga, T. Kurihara, and M. Higashi, "Silicon interposer with TSVs (Through Silicon Vias) and fine multilayer wiring," in *Proc. of ECTC*, Lake Buena Vista, FL, USA, May 2008, pp. 847–852.

[4] Y.-K. Ho and Y.-W. Chang, "Multiple chip planning for chip-interposer codesign," in *Proc. of DAC*, Austin, TX, USA, June 2013, pp. 1–6.

[5] W.-H. Liu, M.-S. Chang, and T.-C. Wang, "Floorplanning and signal assignment for silicon interposer-based 3D ICs," in *Proc. of DAC*, San Francisco, CA, USA, June 2014, pp. 1–6.

[6] E. Nasiri, J. Shaikh, A. Hahn Pereira, and V. Betz, "Multiple dice working as one: CAD flows and routing architectures for silicon interposer FPGAs," *IEEE TVLSI*, vol. 24, no. 5, pp. 1821–1834, 2016.

[7] S. Osmolovskyi, J. Knechtel, I. L. Markov, and J. Lienig, "Optimal die placement for interposer-based 3D ICs," in *Proc. of ASPDAC*, Jeju Island, Korea, January 2018, pp. 513–520.

[8] CoWoS®—Taiwan Semiconductor Manufacturing Company Limited—TSMC. (n.d.). [Online]. Available: https://3dfabric.tsmc.com/english/dedicatedFoundry/technology/cowos.htm

[9] Fan-out chip on substrate. ASE. (n.d.). [Online]. Available: https://ase.aseglobal.com/public/en/technology/focos.html

[10] R. Mahajan, R. Sankman, N. Patel, D.-W. Kim, K. Aygun, Z. Qian, Y. Mekonnen, I. Salama, S. Sharan, D. Iyengar, and D. Mallik, "Embedded multi-die interconnect bridge (EMIB)—a high density, high bandwidth packaging interconnect," in *Proc. of ECTC*, Las Vegas, NV, USA, May 2016, pp. 557–565.

[11] R. Mahajan, Z. Qian, R. S. Viswanath, S. Srinivasan, K. Aygün, W.-L. Jen, S. Sharan, and A. Dhall, "Embedded multidie interconnect bridge—a localized, high-density multichip packaging interconnect," *IEEE TCPMT*, vol. 9, no. 10, pp. 1952–1962, 2019.

[12] R. Mahajan and S. Sane, "Microelectronic package containing silicon patches for high density interconnects, and method of manufacturing same," USA Patent 8 064 224, 2011.

[13] R. Starkston, D. Malik, J. S. Guzek, C.-P. Chiu, D. Kulkarni, and R. V. Mahajan, "Localized high density substrate routing," USA Patent 9 136 236, 2015.

[14] H. Braunisch, A. Aleksov, S. Lotz, and J. Swan, "High-speed performance of silicon bridge die-to-die interconnects," in *Proc. of EPEPS*, San Jose, CA, USA, October 2011, pp. 95–98.

[15] A. C. Durgun, Z. Qian, K. Aygun, R. Mahajan, T. T. Hoang, and S. Y. Shumarayev, "Electrical performance limits of fine pitch interconnects for heterogeneous integration," in *Proc. of ECTC*, Las Vegas, NV, USA, May 2019, pp. 667–673.

[16] T.-C. Chen and Y.-W. Chang, "Modern floorplanning based on B*-tree and fast simulated annealing," *IEEE TCAD*, vol. 25, no. 4, pp. 637–650, 2006.

[17] Y.-C. Chang, Y.-W. Chang, G.-M. Wu, and S.-W. Wu, "B*-trees: a new representation for non-slicing floorplans," in *Proc. of DAC*, Los Angeles, CA, USA, June 2000, pp. 458–463.

[18] T. Ma and E. F. Y. Young, "TCG-based multi-bend bus driven floorplanning," in *Proc. of ASPDAC*, Seoul, Korea, March 2008, pp. 192–197.

[19] J.-M. Lin and Y.-W. Chang, "TCG: a transitive closure graph-based representation for non-slicing floorplans," in *Proc. of DAC*, Las Vegas, NV, USA, June 2001, pp. 764–769.

[20] I.-P. Wu, H.-C. Ou, and Y.-W. Chang, "QB-trees: towards an optimal topological representation and its applications to analog layout designs," in *Proc. of DAC*, Austin, TX, USA, June 2016, pp. 1–6.

[21] H.-F. Tsao, P.-Y. Chou, S.-L. Huang, Y.-W. Chang, M. P.-H. Lin, D.-P. Chen, and D. Liu, "A corner stitching compliant B*-tree representation and its applications to analog placement," in *Proc. of ICCAD*, San Jose, CA, USA, November 2011, pp. 507–511.

[22] W.-H. Liu, M.-S. Chang, and T.-C. Wang, "Floorplanning and signal assignment for silicon interposer-based 3D ICs," in *Proc. of DAC*, San Francisco, CA, USA, June 2014, pp. 1–6.

[23] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms, 3rd Edition*. The MIT Press, 2009, p. 631.

[24] Intel® Stratix® 10 FPGAs overview—high performance Intel® FPGA. Accessed: 2022-11-14. [Online]. Available: https://www.intel.com/content/www/us/en/products/details/fpga/stratix/10/article.html

[25] Intel unveils new tools in its advanced chip packaging toolbox: Intel newsroom: Deutschland. Accessed: 2022-11-14. [Online]. Available: https://newsroom.intel.de/news/intel-unveils-new-tools-in-its-advanced-chip-packaging-toolbox/\#gs.immfzt

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on August 20,2026 at 09:43:43 UTC from IEEE Xplore. Restrictions apply.